



软件体系结构

第4章 数据流风格 ——管道-过滤器

李昶松

南京理工大学计算机科学与工程学院

lixs@njust.edu.cn

软件体系结构风格的定义



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 软件体系结构风格 (software architecture style), 又称软件体系结构惯用模式 (software architecture idiomatic paradigm), 是描述某一特定应用领域中系统组织方式的惯用模式, 作为“可复用的组织模式和习语”, 为设计人员的交流提供了公共的术语空间, 促进了设计复用与代码复用。

Architectural styles help software engineers to reason about architectural qualities.

体系结构风格帮助软件工程师
推断软件体系结构的质量

- A style
 - describes a class of architectures
描述一类体系结构
 - is found repeatedly in practice
在实践中被多次设计、应用
 - is a package of design decisions
是若干设计思想的综合
 - has known properties that permit reuse
具有已经被熟知的特性，并且可以复用

- A style is determined (described) by
 - a set of component types (e.g., data repository, process, object)
一组组件类型（例如：数据容器，过程，对象）
 - a set of connector types/interaction mechanisms (e.g., subroutine call, event, pipe)
一组连接件类型/交互机制（例如：过程调用，事件，管道）
 - a topological layout of these components
这些组件的拓扑分布

- A style is determined (described) by
 - a set of constraints on topology and behavior (e.g., a data repository is not allowed to change stored values, pipelines are acyclic)
一组对拓扑和行为的约束（例如：数据容器不能自己存储数据，管道不能是循环的）
 - an *informal* description of the costs and benefits of the style, e.g.: “Use the pipe and filter style when reuse is desired and performance is not a top priority
一些对风格的成本和益处的 *非正式* 的描述，例如：如果你需要重用性并且性能不是很重要，那么可以使用管道风格

- 使用体系结构风格的优势
 - 它极大地促进了设计的重用性和代码的重用性，并且使得系统的组织结构易被理解。
 - 即使没有给出具体实现细节，依然可以通过“客户/服务器”体系结构风格大致推测出系统的组成结构和工作方式。
 - 使用标准的体系结构风格可较好地支持系统内部的互操作性以及针对特定风格的分析
 - 如：“管道/过滤器”风格可用来分析调度、吞吐量、延迟，和死锁等问题。

Garlan and Shaw compiled a catalog of architectural styles in 1995. Others, such as Buschmann, have augmented this.

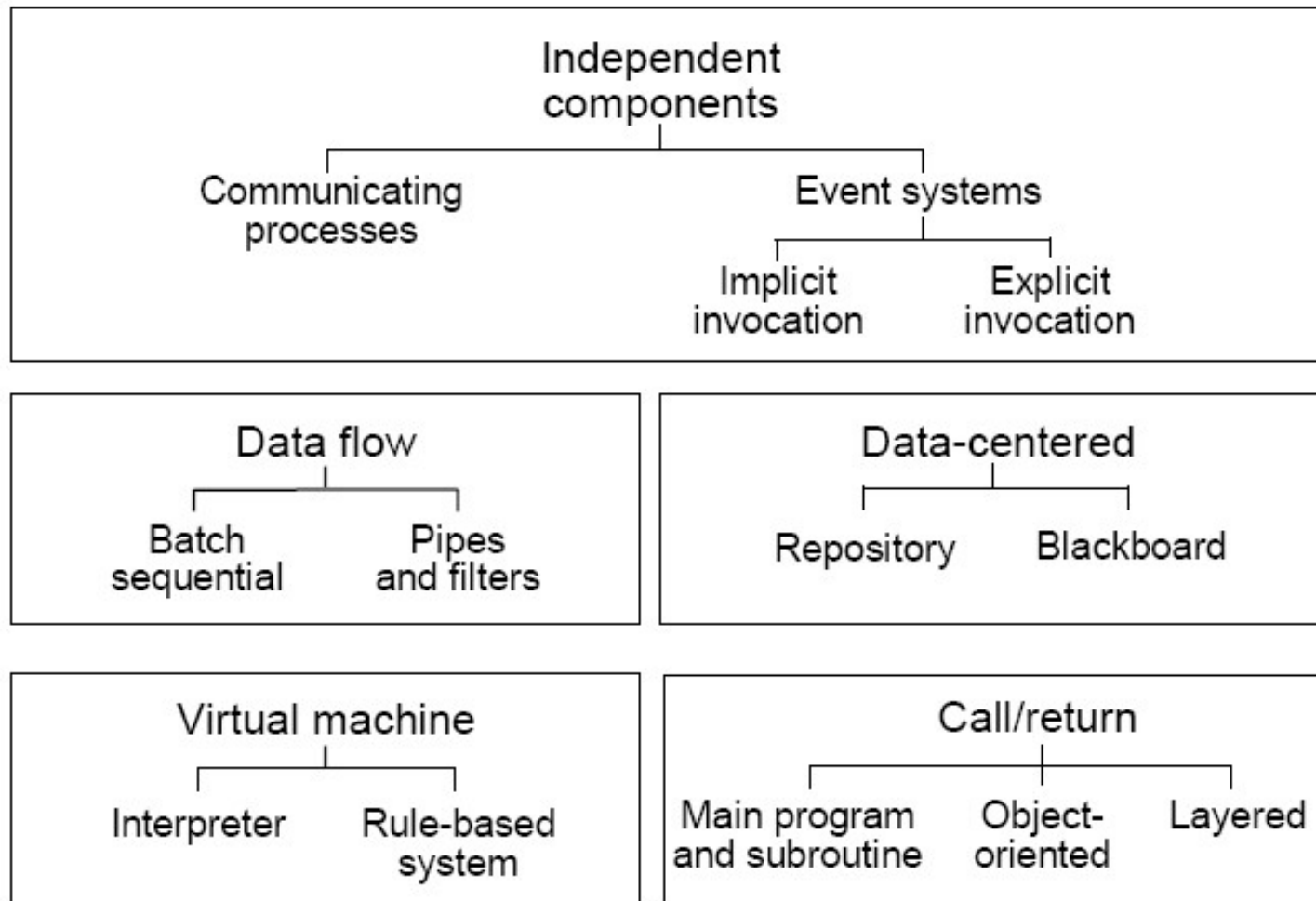
There is no complete list.
没有完备的列表

There is no unique, non-overlapping list.
没有无重叠的列表

Styles overlap. 风格是彼此重叠的

Systems exhibit multiple styles at once.
一个系统通常表现出多种风格

Types of Architectural Styles



- When we introduce a new style, we will typically first examine its “pure” form.
 - pure architectural style are rarely found in practice
纯粹的体系风格在实际中很难遇到
 - systems in practice
 - regularly deviate from the academic definitions of these systems. 循规蹈矩地背离了对这些系统的学术定义
 - typically feature many architectural styles simultaneously
典型地，融合很多体系风格的特色
 - as an architect you must understand the “pure” styles to understand the strength and weaknesses of the style as well as the consequences of deviating from the style
作为一个体系结构师，你必须理解“纯”的风格。理解它的优点与缺陷，也要理解背离此种风格之后会带来什么结果

- A data flow system is one in which
 - the availability of data controls the computation
由数据控制计算
 - the structure of the design is dominated by orderly motion of data from process to process
系统结构由数据在处理之间的有序移动决定
 - the pattern of data flow is explicit
数据流系统的结构是显而易见的
- In a pure data flow system, there is no other interaction between processes
在纯数据流系统中，处理之间除了数据交换，没有任何其他的交互



The Data Flow Style

- There are variety of variations on this genera theme:
 - how control is exerted (e.g., *push* versus *pull*)
如何施加控制（比如：推还是拉）
 - degree of concurrency between processes
并行的程度
 - topology

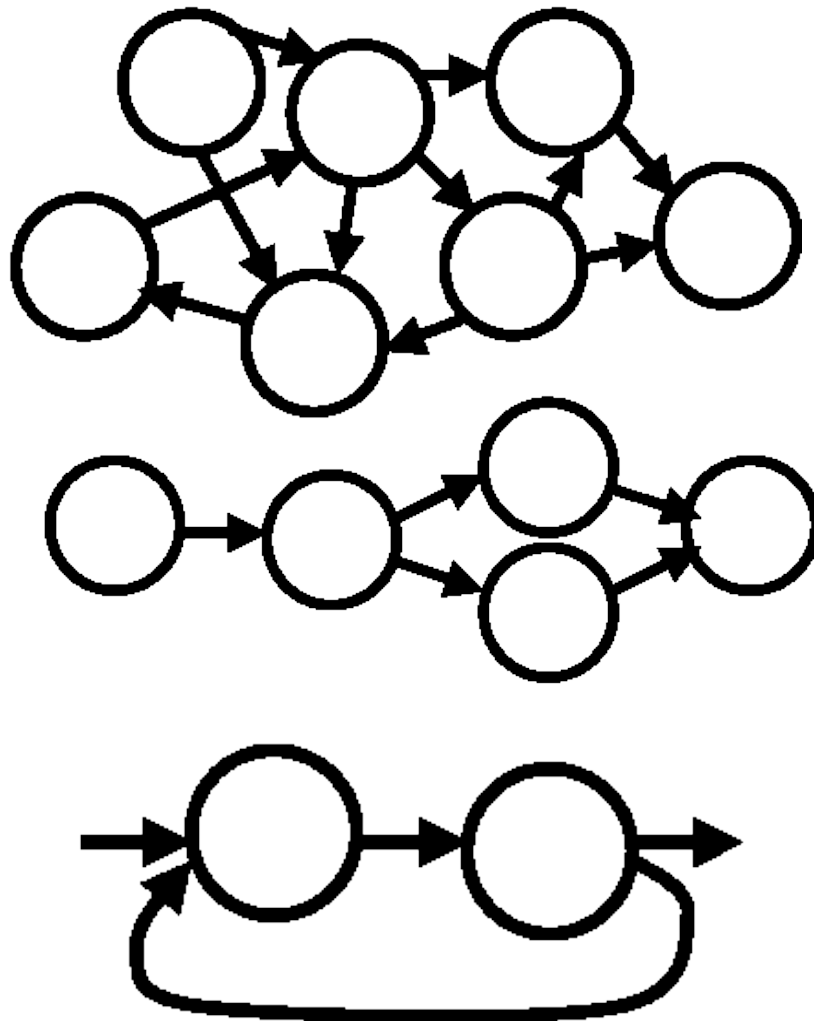
- Components: Data Flow Components
 - Interfaces are input ports and output ports
组件接口是输入端口和输出端口
 - Input ports read data; output ports write data
从输入端口读数据，向输出端口写数据
 - Computational model: read data from input ports, compute, write data to output ports
计算模型：从入口读数，计算，然后写到出口



Data Flow Styles

- Connectors: Data Streams
 - Uni-directional (单向)
 - usually asynchronous, buffered (通常是异步的, 有缓冲)
 - Interfaces are reader and writer roles
接口是reader和writer
 - 计算模型: 把数据从writer转向reader
- Systems
 - Arbitrary graphs (任意拓扑结构)
 - Computational model: functional composition (计算模型: 功能组合)

Patterns of Data Flow in Systems



In general, data can flow in arbitrary patterns（一般情况，数据的流向无序）

Often we are primarily interested in nearly linear data flow systems（我们主要研究近似线性数据流）

or in very simple, highly constrained cyclic structures（或者是在限度内的循环数据流）

Control Flow vs. Data Flow



- Control Flow (typical case in procedural systems)
 - Dominant question is how locus of control moves through the program
主要问题是控制点怎样在程序或系统之间移动
 - Data may accompany the control but is not the driving force
数据可能跟着控制走，但是并不起推动系统运转的作用
 - Primary reasoning is about order of computation
关注的核心是计算顺序

Control Flow vs. Data Flow



- Data Flow

- Dominant question is how data moves through a collection of (atomic) computations
主要问题是数据怎样在运算单元之间流动
- As data moves, control is “activated”
数据到了，控制（计算）单元便开始工作
- We reason about data availability, transformations, latency,...
我们关心数据是否可用，转换，延迟.....

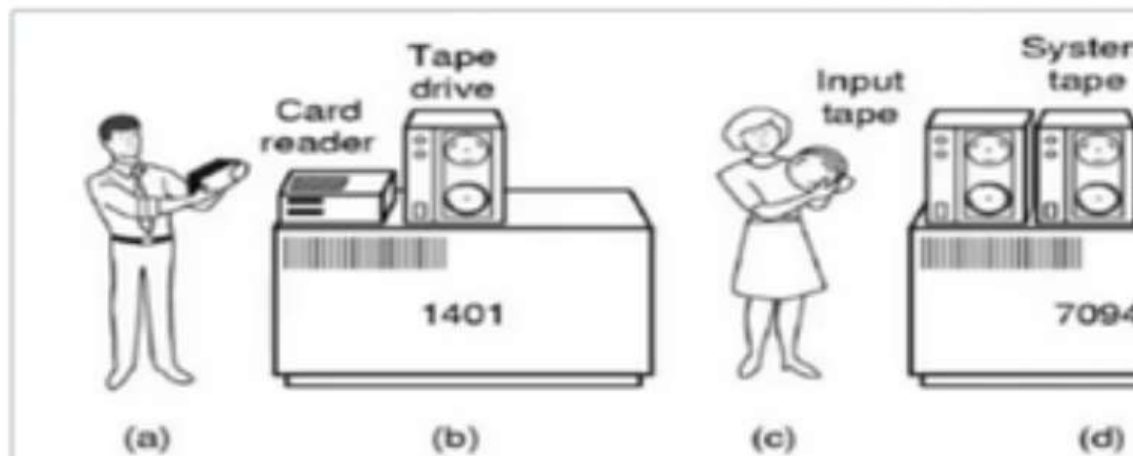
- 数据流风格软件系统被看作
 - 由一些系列作用于数据集合上的“变换”组件组成
 - 这些组件相互独立，可重复使用，可并发执行
 - 不存在全局控制组件的行为
- 软件工程师关注点
 - 数据如何流动
 - 数据的可用性、变换和延迟

Three Examples of Data Flow



- *Batch Sequential* (顺序批处理)
- Pipe-and-Filter (管道过滤器)
- Process Control (控制)

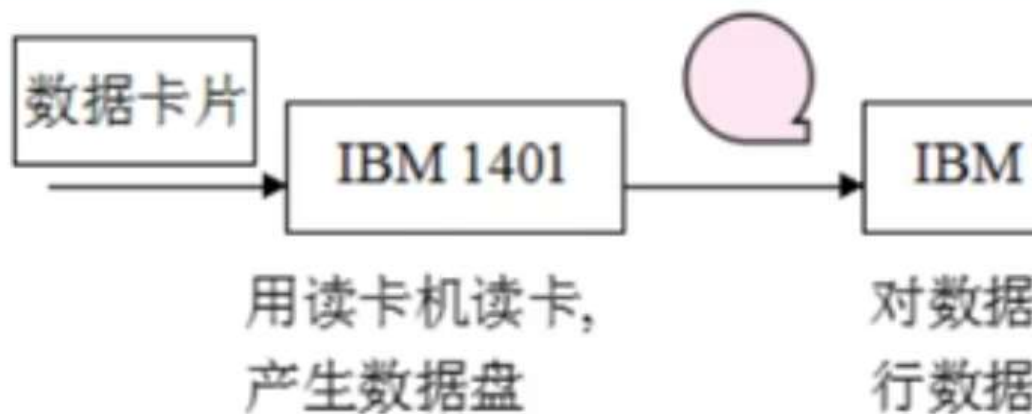
顺序批处理



@工业科技迷

将用户输入的纸带上的
数据写入磁带

将磁带作为计算机的输入
进行计算，

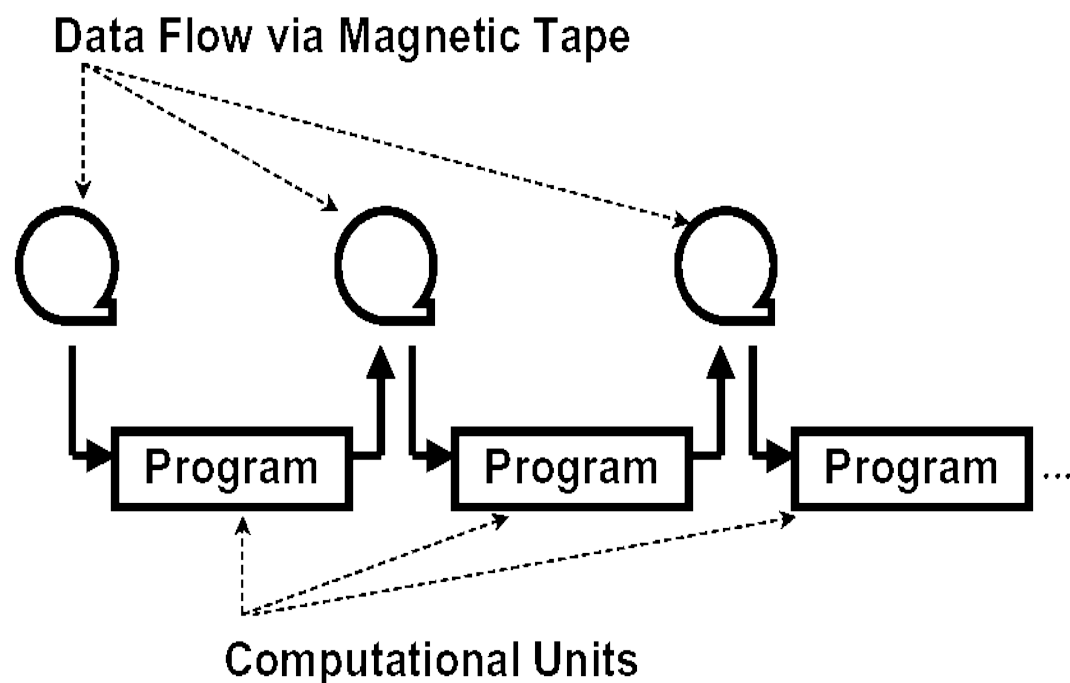


结果打印

顺序批处理定义



- 顺序批处理过程由一些有序的处理步骤组成，这些步骤之间通过存储设备（如磁带、磁盘）衔接在一起
- 每个处理步骤对于输入存储设备上的数据进行一定的处理（可能是获取有用信息或者对数据内容进行修改），并将处理结果写到下游存储设备



- 每个处理步骤是一个独立的程序
- 每一步必须在前一步结束后才能开始
- 数据必须是完整的，以整体的方式传递
- 由于组件顺序执行，不必对组件进行额外的同步处理
- 性能较差
- 不适用于要求对数据进行实时处理的系统



顺序批处理实例

- 更新商店数据库的批处理系统
 - 对于每天形成的数据盘**D1**，进行第一次批处理
Validate，目的是确保数据格式正确，挑选出来格式有错误的记录，并将所有格式正确的记录写入数据盘**D2**
 - 接下来的处理要对**D2**中的内容进行排序，形成新的数据盘**D3**
 - 最后将**D3**的内容插入已经存在的数据盘**D4**





1. 原始数据

Sun, Mike	356821	TV	888.56	324567348
Liu, Elle	672452	356	346126788	camera
Lee, Mary	842674	radio	65.44	563278977
Juell, Paul	257215	Telephone	97.05	678231567
Nygard, Ken	573245	Microwave oven	56.88	974678577
Magel, Ken	834265	refrigerator	988	874267847
Hunt, Paul	332184	Bicycle	523	223566878
Eric, Vosika	257652	jewellery	1005	564310976



2. 确认后的数据

Sun, Mike	356821	TV	888.56	324567348
Lee, Mary	842674	radio	65.44	563278977
Juell, Paul	257215	Telephone	97.05	678231567
Nygard, Ken	573245	Microwave oven	56.88	974678577
Magel, Ken	834265	refrigerator	988	874267847
Hunt, Paul	332184	Bicycle	523	223566878
Eric, Vosika	257652	jewellery	1005	564310976

剔除错误格式数据

Liu, Elle	672452	camera	356	346126788
-----------	--------	--------	-----	-----------



3. 数据排序

Eric, Vosika	257652	jewellery	1005	564310976
Hunt, Paul	332184	Bicycle	523	223566878
Juell, Paul	257215	Telephone	97.05	678231567
Lee, Mary	842674	radio	65.44	563278977
Magel, Ken	834265	refrigerator	988	874267847
Nygard, Ken	573245	Microwave oven	56.88	974678577
Sun, Mike	356821	TV	888.56	324567348

4. 将数据插入主数据库



顺序批处理面临的问题

- 面临的技术压力：在线存取
- 在线存取的优势：
 - 查询更为容易
 - 实时事务处理
- 要实现在线存取、更新需要由批处理方式向交互式处理转换



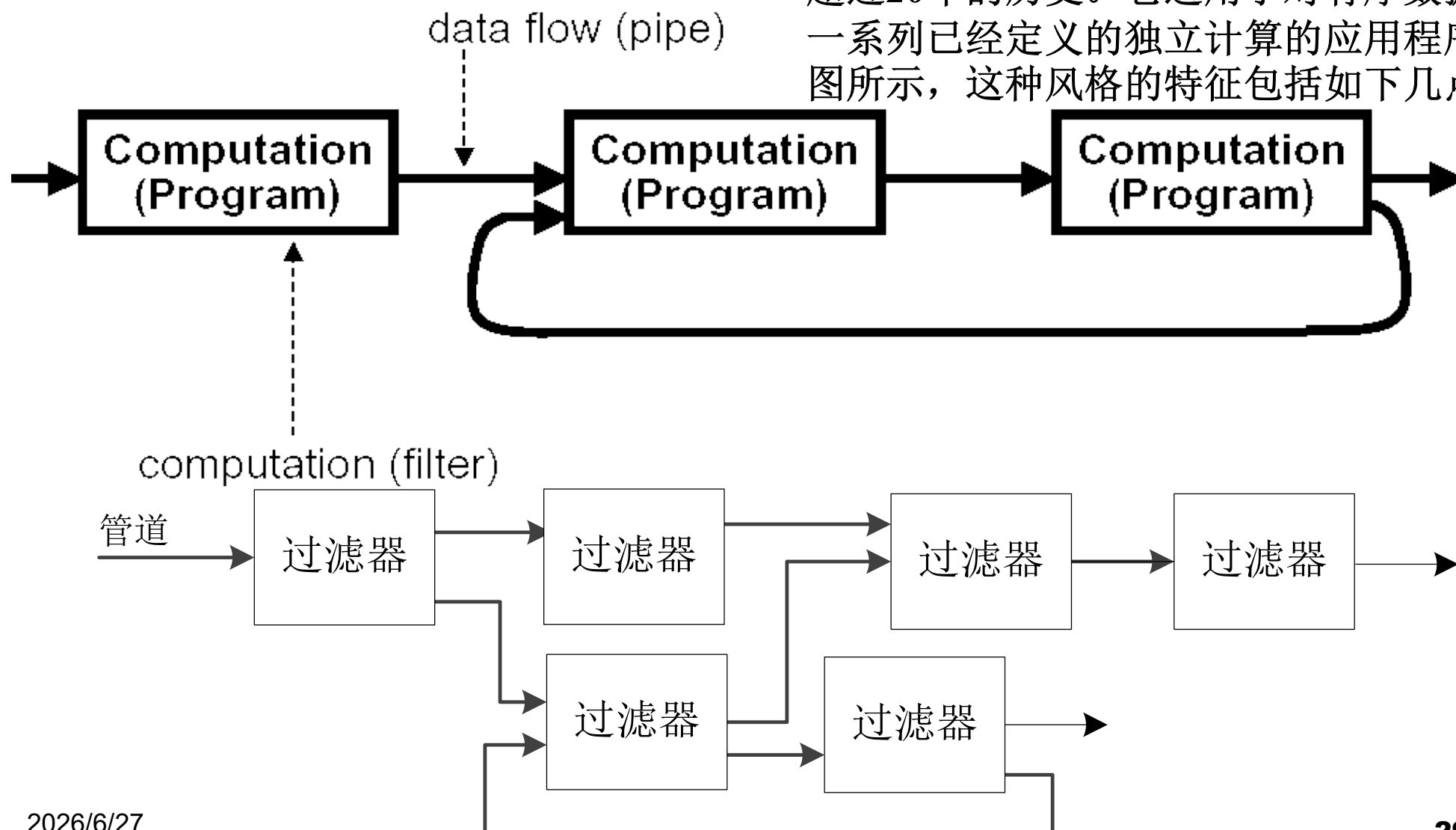
Three Examples of Data Flow

- Batch Sequential (顺序批处理)
- *Pipe-and-Filter* (管道过滤器)
- Process Control (控制)

管道过滤器



管道过滤器风格最早在Unix中出现，已经有超过20年的历史。它适用于对有序数据进行一系列已经定义的独立计算的应用程序。如图所示，这种风格的特征包括以下几点。

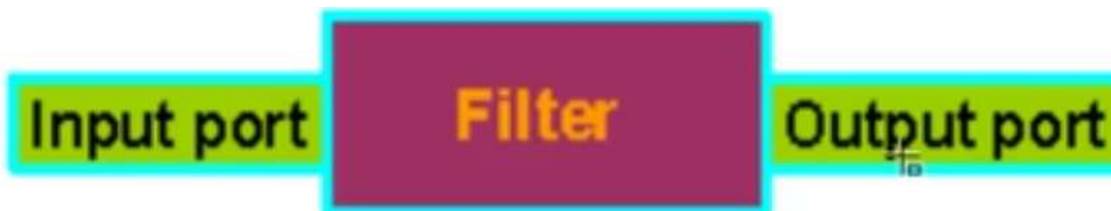


管道过滤器



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

A filter



处理数据

A pipe



传输数据

管道过滤器风格的特征:



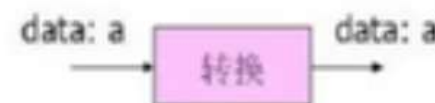
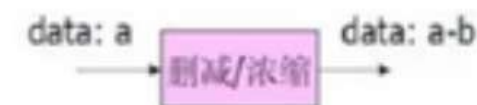
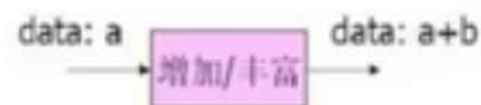
- 构件：在管道过滤器风格中，构件被称为过滤器(filter)。它对输入流进行处理、转换，处理后的结果在输出端流出。而且，这种计算常常是递进的，所以可能在全部的输入接收完之前就开始输出。可以并行地使用过滤器。
- 连接件：连接件位于过滤器之间，起到信息流的导管的作用，被称为管道(pipe)。
- 每个构件都有输入/输出集合，构件在输入处读取数据流，并在输出处生成数据流。
- 过滤器必须是独立的实体，它们不了解信息流从哪个过滤器流出，也不需要知道信息将流入哪个过滤器。它们可以指定输入的格式，可以确保输出的结果，但是它们可能不知道在管道之后将会是什么样子。过滤器之间也不共享状态。

过滤器的功能



- 增量式地把输入数据转为输出数据
 - 通过计算或添加信息丰富数据
 - 改变数据表示形式
 - 流到流的转换
 - 其他

过滤器对数据流的五种变换类型



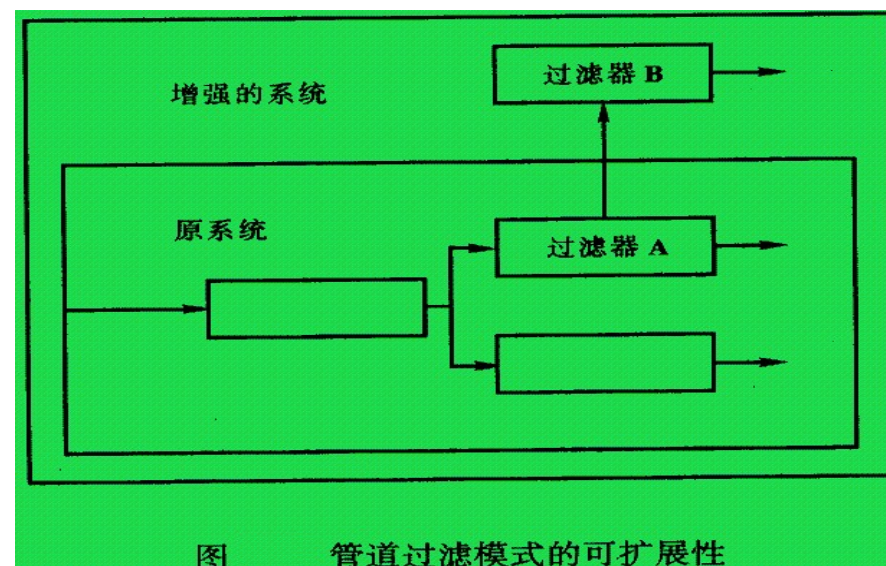
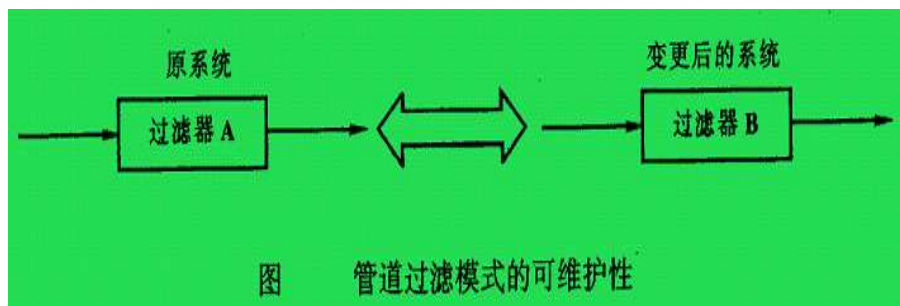
- 主动过滤器：主动从管道拉入（pull）或推出（push）数据
- 被动过滤器：数据流由管道负责推动

- 将数据从上一个过滤器的输出端移动到下一个过滤器的输入端
 - 通常是单向流动
 - 管道连接过滤器形成传输图（拓扑）

- 什么力量推动数据的流动？
 - Push: 数据源向后推出
 - Pull: 数据接收者从前方拉入
 - Push/Pull结合: 过滤器主动拉出, 再将处理后的数据推入

管道过滤器风格的优点

- (1) 由于每个组件行为不受其他组件的影响，整个系统的行为易于理解。
- (2) 支持功能模块的重用，任意两个过滤器只要相互间所传输的数据格式上达到一致，就可以连接在一起。
- (3) 系统易于维护和扩展。新的过滤器容易加入到系统中，旧的过滤器也可被改进的过滤器替换。
- (4) 支持特殊的分析，如吞吐量分析、死锁分析。
- (5) 支持并发执行，每个过滤器既可以独立运行，也可以和其它过滤器并发执行。

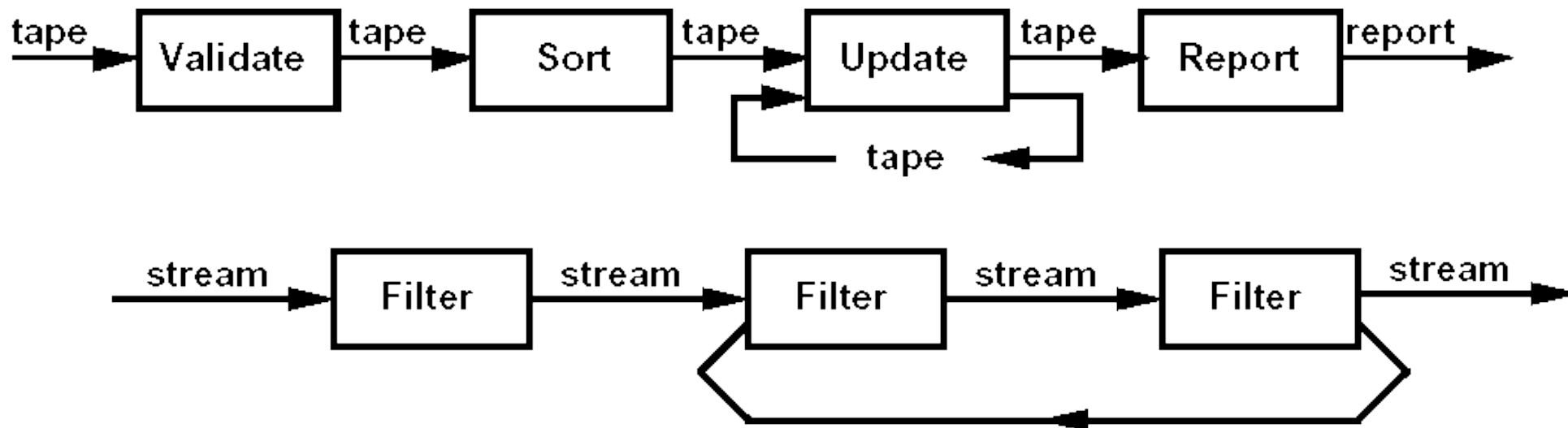


管道过滤器风格的缺点

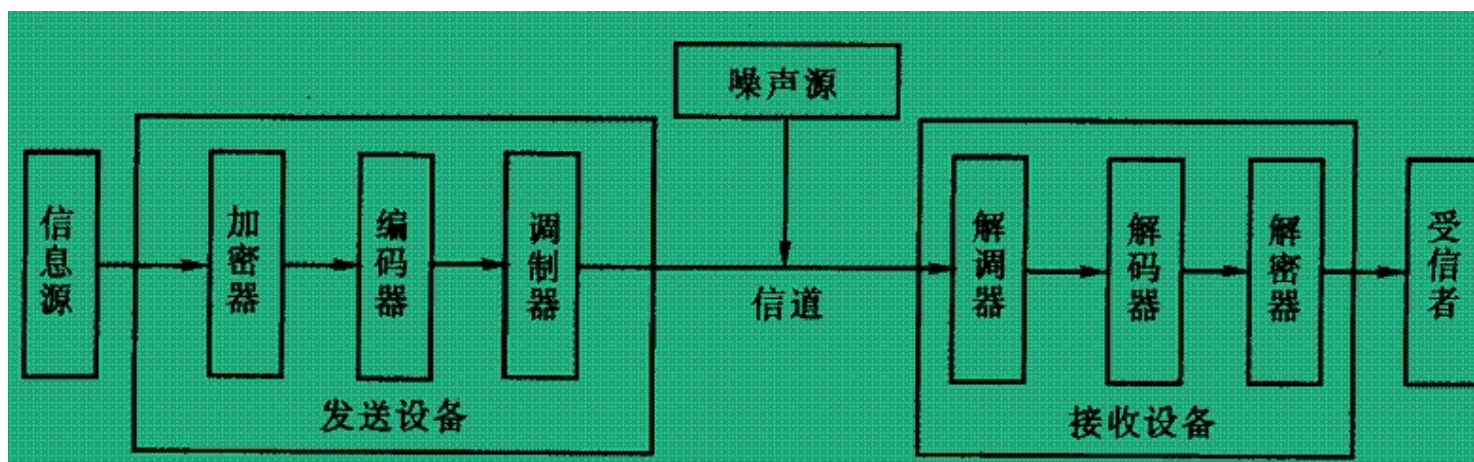
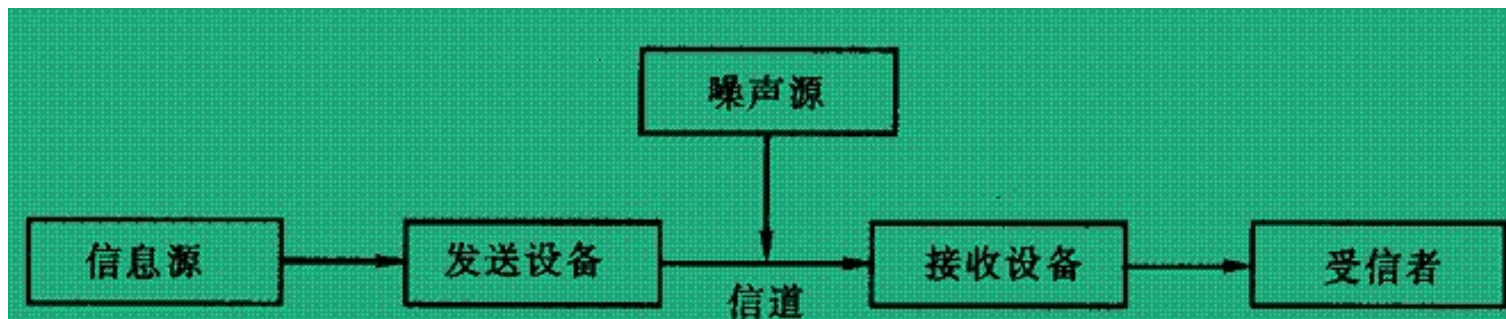


- 往往导致系统处理过程的成批操作。
- 在处理两个独立但又相关的数据流时可能会遇到困难（交互式的应用程序）。
- 需要对数据传输进行特定的处理时，会导致对于每个过滤器的解析输入和格式化输出要做更多的工作，带来系统复杂性的上升。
- 并行处理的效率往往是假象
 - 过滤器之间传输代价、部分过滤器写得很差、进程线程的关联转换成本、同步化

Batch Sequential vs Pipe & Filter



Pipe & Filter

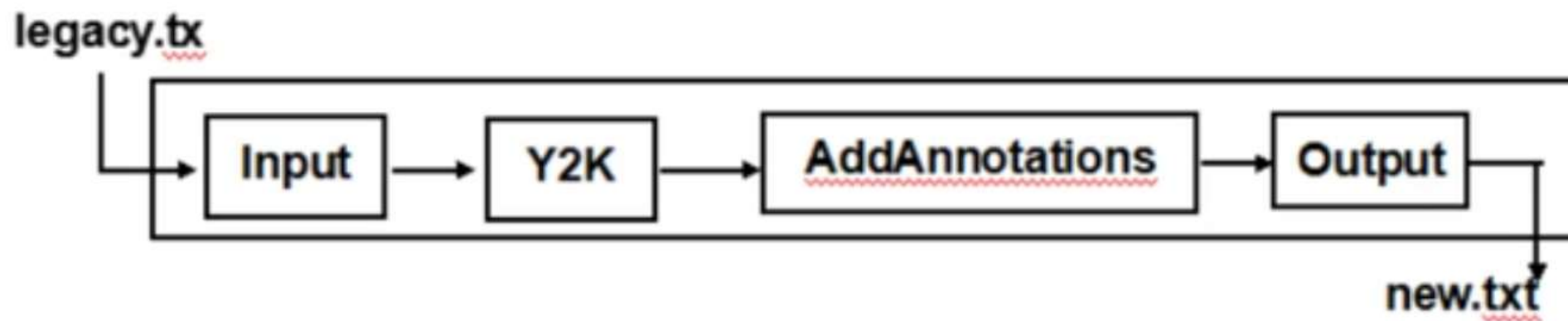




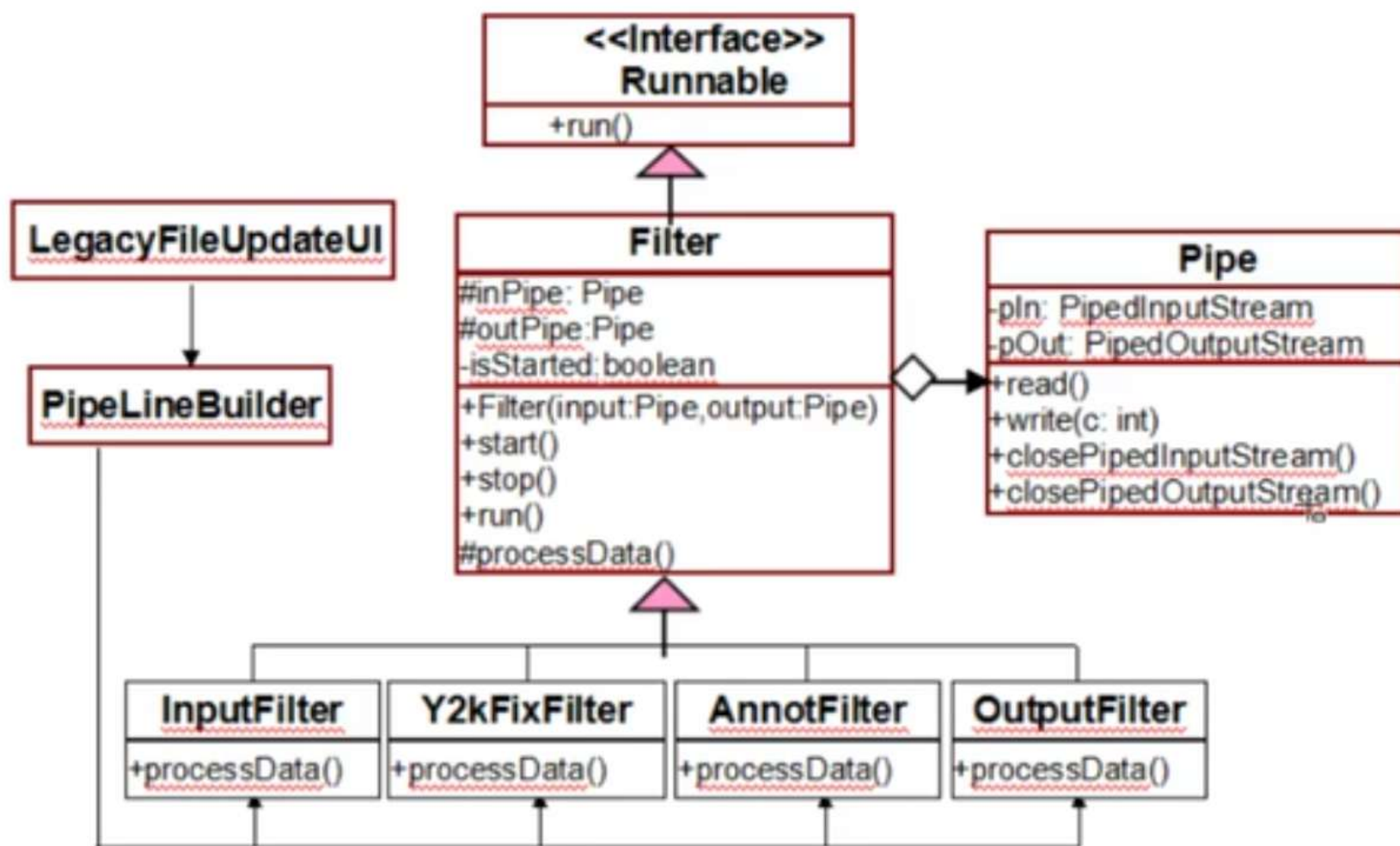
Java语言对管道-过滤器架构的支持

- 所有基本的**Java I/O**都基于流(**stream**)的。一个流代表“数据流”或者一个用于一端带有一个“写”而另外一端带有一个“读”通讯的信道(**channel**)。
- 可以使用**Java**中的管道类**PipedWriter** 和 **PipedReader**组成被动型的管道，写入**PipedWriter**对象的数据，可以由与之相连接的**PipedReader**对象读出。

- 千年虫问题
 - 将2位年份改为4位，如95->1995
 - 为处理完的行添加注释，如处理日期



常见实现方式





UNIX操作系统对管道-过滤器架构的支持

- UNIX中许多命令与管道-过滤器架构相关
- UNIX中通常将标准输入stdin转换为标准输出stdout的进程称为过滤器
- UNIX中通常假设管道中传输的是ASCII字符流
- UNIX中的管道通常将文件视为数据源Data Source和数据接收者Data Sink



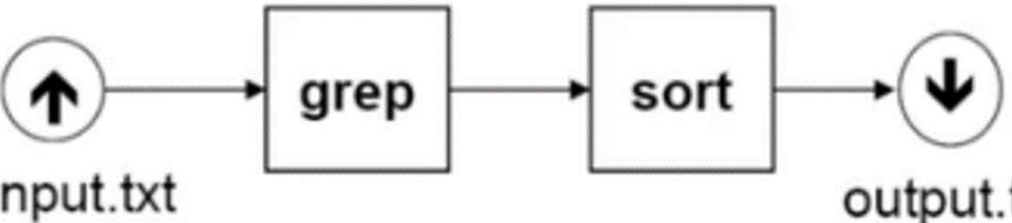
管道-过滤器风格实例

- 在Unix和Dos中，允许在命令中出现用竖线字符“|”分开的多个命令，将符号“|”之前的命令的输出，作为“|”之后命令的输入，这就是“管道功能”，竖线字符“|”是管道操作符。

- 例如： Unix shell中：

- cat file|grep xyz|sort|uniq>out

- 即找到含xyz的行，排序、去掉相同的行，最后输出

- 例如： 
 - 命令dir input.txt 显示。
 - dir的输出，由于符号“|”的规定，成为下一个命令more的输入，more命令则将其输入，more命令则将其输入一屏一屏地显示，成为命令行的输出。

管道-过滤器风格 实例



- dir | more

```
C:\WINDOWS\system32\cmd.exe
C:\>cd windows\system32

C:\WINDOWS\system32>dir | more
驱动器 C 中的卷没有标签。
卷的序列号是 30CC-6AD3

C:\WINDOWS\system32 的目录

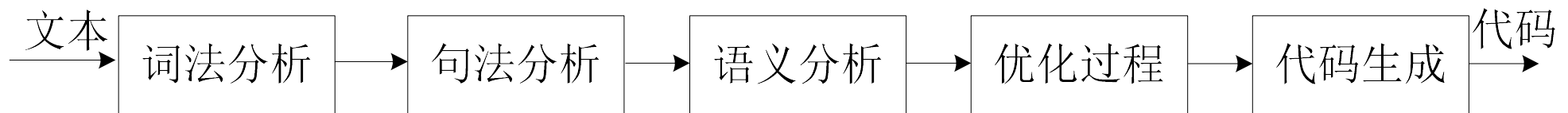
2008-10-17  19:23    <DIR>          .
2008-10-17  19:23    <DIR>          ..
2008-10-09  10:35             378 $winnt$.inf
2008-10-09  18:07    <DIR>          1025
2008-10-09  18:07    <DIR>          1028
2008-10-09  18:07    <DIR>          1031
2008-10-09  18:08    <DIR>          1033
2008-10-09  18:07    <DIR>          1037
2008-10-09  18:07    <DIR>          1041
2008-10-09  18:07    <DIR>          1042
2008-10-09  18:07    <DIR>          1054
2008-04-14  20:00       2,151 12520437.cpx
2008-04-14  20:00       2,233 12520850.cpx
2008-10-09  18:09    <DIR>          2052
2008-10-09  18:07    <DIR>          3076
2008-10-09  18:07    <DIR>          3com_dmi
2008-04-14  20:00     100,352 6to4svc.dll
2008-04-14  20:00       1,460 a15.tbl
2008-04-14  20:00     44,370 a234.tbl
-- More --
```

管道-过滤器风格实例



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 实例：
 - 传统的编译器，一个阶段的输出是另一个阶段的输入（包括词法分析、语法分析、语义分析和代码生成）



Summary: Guidance for Selecting Data Flow Style



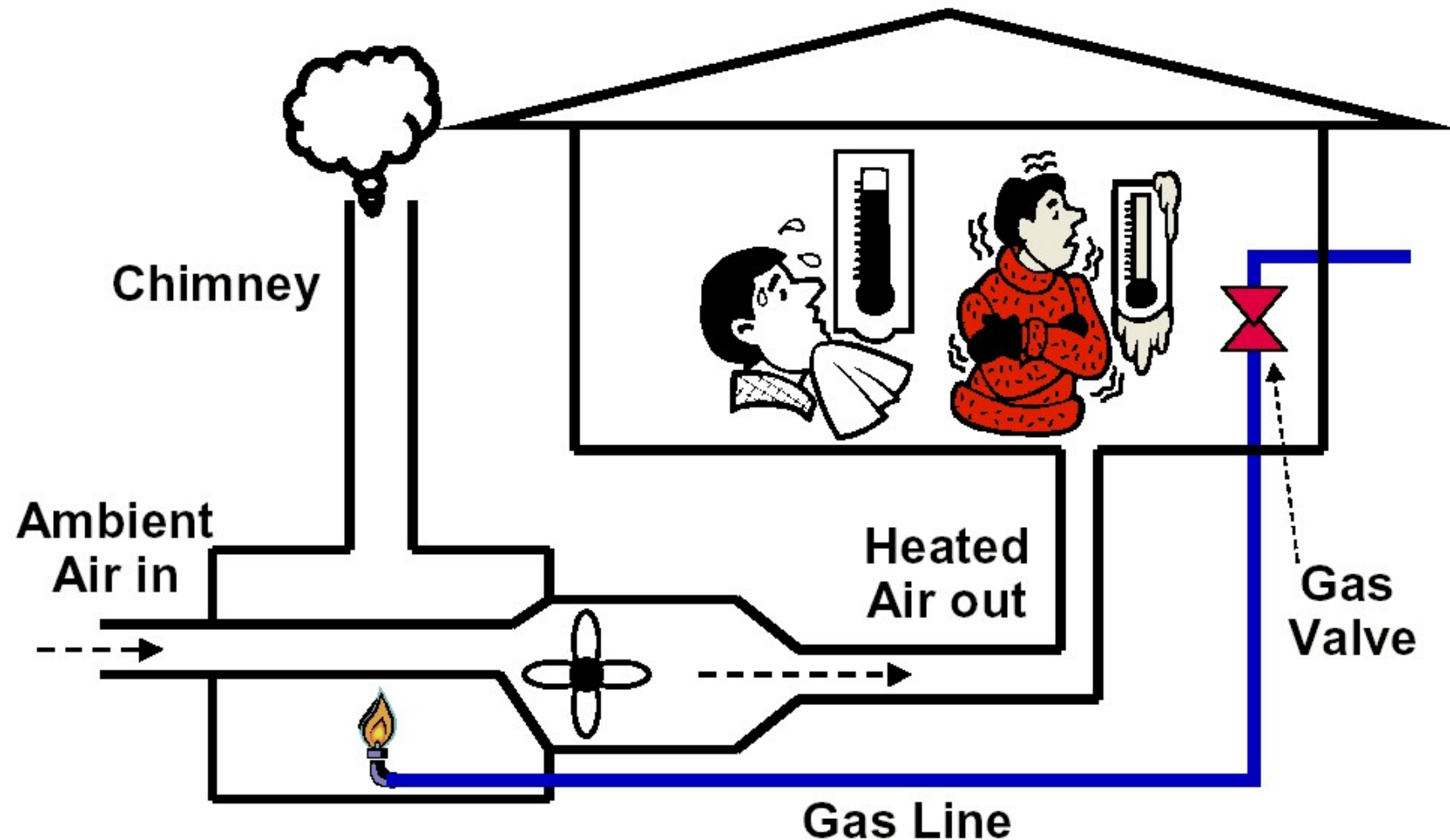
- task is dominated by the availability of data
任务由数据主导
- data can be moved predictably from process to process
事先知道数据的确切流向
- pipe-and-filter architectures are good choices for many data flow applications because
 - they permit reuse and reconfiguration of filters
 - generally easy to reason about
 - reduce system testing
 - may allow incremental AND parallel processing
- there may be a performance penalty in data flow styles, however this depends on many factors
数据的流动带来性能损耗



Three Examples of Data Flow

- Batch Sequential (顺序批处理)
- Pipe-and-Filter (管道过滤器)
- *Process Control (控制)*

Open Loop Control(开环控制)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

